

Task 1

1.1 Project Description

York university students interact with eClass across multiple courses every term. Each course contains messy categorization of information like announcements, forum posts, assignment descriptions, quizzes, lecture slides, and links. Students have to manually read through dozens of announcements and reorient themselves to traverse through each course. This becomes time consuming and prone for students to interpret things incorrectly. As a result, students miss deadlines, priorities, and waste time looking for information.

The targeted users are YorkU students enrolled in multiple eClass courses per term. Also, students with accessibility needs who benefit from simple concise summaries, priorities ranking, language interaction, etc.

The AI agent will take in course materials and, ranks deadlines and assessments by priority, detects what changed, generates study plans, answers questions about the course. All features are reachable from both GUI and CLI.

The AI layer will be using Qwen3.6 via cloud API and qwen3.5 with Ollama for local/offline runs, with embeddings for RAG over slides and PDFs.

The LLM is gonna be a single-agent system that interacts with a system that drives the LLM and uses a set of tools like File reader, PDF parser, etc. There is also a background component that watches for new items between each session.

1.2 Feature Specification

F01 - Deadline & Assessment Summarizer (Hybrid)

- The user opens the “Deadlines” tab and selects a course. The agent reads through course pages, syllabus, and announcements, it then extracts deadlines, weights, and formats, and produces a list. The list is displayed in the GUI and can be edited by the user to correct any errors.

F02 - File change detector (Hybrid)

- The user can toggle a “Watch” button on a course. It will then periodically check on the uploaded content when the program is open, or update the course content when the program is opened again. The log of new content is displayed in the GUI.

F03 - Task priority ranker (AI-based)

- The user can sort the deadlines tab and rank them with a score. The deadlines tab is then sorted with the new ranked list.

F04 - Progress Tracker (Hybrid)

- The user marks tasks as complete via checkboxes in the GUI. The agent updates progress scores and re prioritises remaining tasks.

F05 - Final grade calculator (Deterministic)

- The user opens the “Grades” tab and enters scores per assessment. The system will compute the total grade and output current average. The user can also have a grade goal where the agent can highlight grades needed per assessment to reach a certain final grade.

F06 - Language Q&A (AI-based)

- The user can type questions in the chat panel. The agent embeds the questions and retrieves relevant chunks from stored course materials, and generates an answer. The answer is displayed in the GUI.

F07 - Study plan generator (AI-based)

- The user can set their availability in the “Study” tab and generate a plan. The agent turns the tasks list into subtasks and estimates the time needed to complete. The plan is displayed in the GUI as a schedule and can be adjusted.

F08 - Slides Summary / Notes (AI-based)

- The user can upload or sync lecture slides and click “notes”. The agent reads the file, extracts key information, and generates structured study notes in a certain style. The notes are displayed in the GUI.

F09 - Weekly Course report (AI-based)

- The user can click “generate report”. The agent gathers outputs from exams, deadlines, grades, and puts them in a report. The report is displayed in the GUI and can be exported as a PDF.

F10 - Practice quiz generator (AI-based)

- The user opens the “practice” tab, selects a course, topic, and lecture notes. The agent retrieves relevant materials, generates practice questions, and grades the user’s answers. The quiz and results are displayed in the GUI.

Task 2

Pattern 1 - Template Method

Different AI features share common workflow, gather materials, gather assessments, set up the LLM, set up embeddings, execute the agent, process results. However, each feature has different specific steps.

Class	Role
AITemplate	Abstract Class - defines execute(), getMaterials(), getAssessments(), setEmbed(), setLLM()
Agent	Participant - used by AITemplate to execute prompts
llmSource	Participant - to set LLM via setLLM()
EmbeddingTool	Participant - to set embed via setEmbed()

The AITemplate class defines a skeleton of the controllers algorithm. Concrete classes can override specific steps like material fetching or prompt uses. This allows code to be reused and the ease of adding more features.

Without the template method, each AI feature would need to duplicate the entire workflow, gather materials, gather assessments, set up the LLM, set up embeddings, execute the agent, process results. This would lead to code duplication and difficulty to maintain when the workflow changes.

Pattern 2 - Strategy

The application needs to support multiple LLM and embedding providers. Users may want to switch between cloud-based and local AI providers depending on connectivity or preference

Class	Role
llmSource	Strategy Interface - defines run(context: String): String
EmbeddingProvider	Strategy Interface - defines embed(context: String): float[]
VectorStore	Strategy Interface - defines delete(), query(), upsert()
CloudQwenSource	Concrete - Cloud LLM
CloudEmbedder	Concrete - Cloud embedding
CloudVectorStore	Concrete - Cloud vector storage
LocalQwenSource	Concrete - Local LLM
LocalEmbedder	Concrete - Local embedding
LocalVectorStore	Concrete - Local vector storage

Agent	Context - uses llmSource
AITemplate	Context - holds reference to llmSource and EmbeddingTool
EmbeddingTool	Context - uses EmbeddingProvider and VectorStore

The application has to support both online and offline AI features. The Strategy pattern allows the system to swap between cloud and local LLM at runtime without modifying the Agent or AITemplate classes. The embedding and vector storage can also be swapped independently

Pattern 3 - Observer

When course material is uploaded or an assessment is completed, multiple components need to react. The MaterialTracker updates the material records, and the AssessmentTracker updates the assessment statuses. The Course class needs to notify these observers.

Class	Role
CourseListener	Observer Interface - declares update()
MaterialTracker	Concrete - updates when materials change
AssessmentTracker	Concrete - updates when assessments change
Course	Subject - maintains observers, addObserver(), removeObservers(), notify()

Pattern 4 - Factory

The application needs to create families of AI components that must be compatible with each other. Cloud setup should use cloud components and a Local setup should use local components.

Class	Role
AIFactory	Abstract Factory - declares createLLMsource(), createEmbeddings(), createVectorStore()
OnlineFactory	Concrete - CloudQwenSource, CloudEmbedder, CloudVectorStore
OfflineFactory	Concrete - LocalQwenSource, LocalEmbedder, LocalVectorStore
llmSource	Abstract Product
EmbeddingProvider	Abstract Product
VectorStore	Abstract Product
CloudQwenSource	Concrete product

LocalQwenSource	Concrete product
CloudEmbedder	Concrete product
LocalEmbedder	Concrete product
CloudVectorStore	Concrete product
LocalVectorStore	Concrete product

The Abstract Factory ensures that when a user selects “online”, they will get a family of cloud components. When the user selects “offline”, they will get a family of local components. This prevents any mismatched configurations.

Without Abstract Factory, the client code would need to instantiate each component individually and have to check compatibility. This could lead to configuration errors. Adding a new LLM would change throughout many parts rather than adding a new factory.

Pattern 5 - Facade

The application has many complex subsystems. The GUI and CLI needs a simplified interface to interact with all these subsystems without knowing the systems.

Class	Role
AppFacade	Facade - provides simplified methods
GUI	Client - uses facade
CLI	Client - uses facade
DeadlineController TaskController GradeCotnroller LanguageQAControll er StudyController NotesController ReportController QuizController	SubSystems
AITemplate Agent Storage	SubSystems

Both GUI and CLI need to perform the complex operations. The Facade provides a single entry point that coordinates multiple controllers, AI templates, and storage. This makes the presentation layer less complex.

Without Facade, GUI and CLI would need to directly interact with many controllers, manage, and coordinate them. This would result in duplicated logic between GUI and CLI, and harder to test. Changes to the subsystem would affect all the way up to GUI and CLI.

Task 3

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Manage Courses (create, view, update, delete)	Deterministic	UC1: Manage Courses	GUI AppFacade Course	addCourse() create() getCourse() updateCourse() deleteCourse()	seqUC01	Facade
F02	Upload Course Materials (PDF, slides, links)	Hybrid	UC2: Upload Course Materials	GUI AppFacade Course PDFReaderTool FileReaderTool Materials MaterialTracker	uploadFile() execute() read() notify() update()	seqUC02	Observer Strategy Facade
F03	Track Assessments (view deadlines, status)	Deterministic	UC3: Track Assessments	GUI AppFacade Course Assessment AssessmentTracker	getDeadlines() getAssessments() execute() update()	seqUC03	Observer Facade
F04	Complete Assessments (mark done, record grade)	Deterministic	UC4: Complete Assessments	GUI AppFacade GradeController Course Assessment AssessmentTracker	completeAssessment() execute() complete() notify() update()	seqUC04	Observer Facade
F05	Compute Course Grade (weighted average)	Deterministic	UC5: Compute Grade	GUI AppFacade GradeController GradeReport Grade	computeGrade() execute() calculateAverage() calculateGrade()	seqUC05	Facade Strategy
F06	Generate Grade Report (AI-based)	Hybrid	UC6: Generate Report	GUI AppFacade ReportController AITemplate Agent llmSource	generateReport() execute() getMaterials() getAssessments() setLLM() execute(prompt, data, llm) run(context)	seqUC06	Template Method Strategy Facade
F07	Rank Deadlines by Priority	Hybrid	UC7: Rank Deadlines	GUI AppFacade TaskController DeadlineController	rankDeadlines() execute(deadlines) createAssessments()	seqUC07	Facade Strategy

				r Assessment	calculatePriorityScore()		
F08	Generate Study Plan (AI-based)	Hybrid	UC8: Generate Study Plan	GUI AppFacade StudyController AITemplate Agent llmSource	generatePlan() execute() getAssessments() execute(prompt, data, llm) run(context)	seqUC08	Template Method Strategy Facade
F09	Generate Quiz from Materials (AI-based)	Hybrid	UC9: Generate Quiz	GUI AppFacade QuizController AITemplate Agent llmSource	generateQuiz() execute() getMaterials() execute(prompt data, llm) run(context)	seqUC09	Template Method Strategy Facade
F10	Ask Question about Materials (RAG-based QA)	Hybrid	UC10: Ask Question	GUI AppFacade LanguageQAController AITemplate EmbeddingTool VectorStore Agent llmSource	answerQuestion() execute() embed() query() execute(prompt, data, llm) run(context)	seqUC10	Template Method Strategy Facade
F11	Generate Notes (AI-based)	Hybrid	UC11: Generate Notes	GUI AppFacade NotesController AITemplate Agent llmSource	generateNotes() execute() getMaterials() execute(prompt, data, llm) run(context)	seqUC11	Template Method Strategy Facade
F12	Set Course Goal	Deterministic	UC12: Set Course Goal	GUI AppFacade GradeReport	setGoal() getGoal() calculateAverage()	seqUC12	Facade
F13	Configure AI Source (online/offline)	Hybrid	UC13: Configure AI Source	GUI AppFacade OnlineFactory OfflineFactory CloudQwenSource LocalQwenSource CloudEmbedder LocalEmbedder CloudVectorStore LocalVectorStore AITemplate	switchMode() createLlmSource() createEmbeddings() createVectorStore(), setLLM() setEmbed()	seqUC13	Abstract Factory Strategy Facade

F01 - Manage Courses

Related Sequence Diagram: seqUC01 - Manage Courses

Classes Involved:

- GUI - collects course name and ID from the user and displays the result
- AppFacade - coordinates the request and delegates to Course
- Course - represents the course entity and handles the creation logic

Important Methods:

- GUI.createCourse()
- AppFacade.addCourse(name,id)
- Course.create()

Execution:

- When the user selects "Create Course" in the GUI, it is sent to AppFacade. AppFacade calls Course.create(). If course ID is unique a new Course object is created and returned. If the ID already exists, Course ejects the request and AppFacade returns with an error message.

F02 - Upload Course Materials

Related Sequence Diagram: seqUC02 - Upload Course Materials

Classes Involved:

- GUI - lets the user select a file to upload
- AppFacade - coordinates the upload request
- Course - owns the materials and manages the upload
- PDFReaderTool - reads PDF files
- FileSystem - provides raw file
- MaterialTracker - observes course changes and updates
- Materials - represents the upload material

Important Methods:

- GUI.uploadFile()
- AppFacade.uploadFile()
- Course.uploadFile(filepath)
- PDFReaderTool.execute(s)
- FileSystem.read(path)
- MaterialTracker.update()

Execution:

- When the user selects a file, `uploadFile()` is sent to `AppFacade` which calls `Course.uploadfile`. `Course` calls `PDFReaderTool.execute` to read the file content. The tool calls `FileSystem.read` to retrieve the raw data and returns the content. `Course` creates a `Materials` object and notifies the `MaterialTracker`, which triggers update. `Course` returns success to `AppFacade` and GUI confirms upload to the user. If the PDF fails an error message is returned.

F03 - Track Assessments

Related Sequence Diagram: `seqUC03 - Track Assessments`

Classes Involved:

- GUI - displays the assessment list to the user
- AppFacade - coordinates the request
- Course - holds the assessments
- AssessmentTracker - observes and updates assessment tracking

Important Methods:

- `GUI.viewAssessments()`
- `AppFacade.getDeadlines(courseId)`
- `Course.getAssessments()`
- `AssessmentTracker.execute(s)`
- `AssessmentTracker.update()`

Execution:

- When the user selects “view assessments” `getDeadlines()` is sent to `AppFacade`. `AppFacade` calls `Course.getAssessments()` to retrieve the list. `Course` calls `AssessmentTracker.execute`, which triggers `update()` from the tracker. The tracker returns the list of assessments, which is passed back through `AppFacade` to GUI for display. If no assessment exists, an empty list is returned and a “no assessments” message is returned.

F04 - Complete Assessments

Related Sequence Diagram: `seqUC04 - Complete Assessments`

Classes Involved:

- GUI - lets the user mark an assessment as complete
- AppFacade - coordinates the request
- GradeController - encapsulates the completion request
- Course - owns the assessment and handles completion
- Assessment - represents the assessment being completed
- AssessmentTracker - observes and updates assessment tracking

Important Methods:

- `GUI.completeAssessment()`
- `AppFacade.completeAssessment(a)`

- GradeController.execute()
- Course.completeAssessment(a)
- Assessment.complete()
- AssessmentTracker.update()

Execution:

- When the user marks an assessment as complete, completeAssessment() is sent to AppFacade, which calls GradeController.execute(). The controller calls Course.completeAssessment(a), which calls Assessment.complete(). The AssessmentTracker is notified triggering update(). The grade is returned through GradeController to AppFacade and GUI updates the display.

F05 - Compute Course Grade

Related Sequence Diagram: seqUC05 - Compute Course Grade

Classes Involved:

- GUI - lets the user request grade computation
- AppFacade - coordinates the request
- GradeController - encapsulates the grade computation request
- GradeReport - holds the assessments and computes the average
- Grade - represents individual grade values and calculations

Important Methods:

- GUI.computeGrade()
- AppFacade.computeGrade(courseId, score)
- GradeController.execute()
- GradeReport.calculateAverage()
- Grade.calculateGrade(score, weight)